श्रीचन्द्रशेखरेन्द्रसरस्वतीविश्वमहाविद्यालयः

SRI CHANDRASEKHARENDRA SARASWATHI VISWA MAHAVIDYALAYA

(Accredited with 'A' Grade by NAAC)

(Deemed to be University under section 3 of the UGC act 1956) Enathur,

Kanchipuram, Tamil Nadu - 631 561

FUNDAMENTALS OF MACHINE LEARNING LAB

Name:P.Hyndavi

Reg. No:11259A446

Year & Sem: II Year & III Sem

Section: S8

Department: CSE – AI & ML

Faculty in-charge: Radhika

Subject: Fundamentals of Machine Learning

11259A446

S.NO	Date	Title	Teacher's signature
1.	27/07/26	Basic python programs.	
2.	27/07/26	String programs.	
3.	27/07/26	List programs.	
4.	03/08/26	Dictionary programs	
5.	03/08/26	a) creating array b) Array operation c) Indexing slicing.	
6	10/08/26	pandas (python Data Analysis library)	
7	17/08/26	Matplotlib (plotting library)	
8	14/09/26	Data processing in given set	
9.	14/09/26	Data split up & cross validation.	
10.	14/09/26	Implement linear regression	
11.	21/09/26	Implement classification model using KNN and compute Evaluation.	
12	21/09/26	Implement k-means clustering	
13	21/09/26	Implement hierarchical clustering	
14	28/09/26	Implement random forest and Analyse performance	
15	28/09/26	Implement Boosting technique (Adaboost) & Analysis performance	
16.	05/10/26	Implement a basic reinforcement learning model.	

28/10/26

List Of Experiments

1. Perform data preprocessing on a given dataset.
2. Implement dataset splitting and cross-validation.
3. Implement Linear Regression and evaluate performance.
4. Implement a classification model (KNN/SVM) and compute evaluation metrics.
5. Implement K-Means clustering.
6. Perform hierarchical clustering and compare results.
7. Implement Random Forest and analyze performance.
8. Implement boosting techniques and compare results.
9. Implement a basic reinforcement learning model
10. 10. Build a neural network using TensorFlow/Keras.

1. Perform data preprocessing on a given dataset.

Aim:

To perform **data preprocessing** on a given dataset by handling missing values, selecting numerical attributes, and normalizing the data using **StandardScaler**.

Algorithm:

1. Import the required Python libraries.
2. Load the dataset from `data.csv`.
3. Display the first five rows of the original dataset.
4. Check the dataset for missing values.
5. Replace missing numerical values with their respective column means.
6. Select the numerical columns: `Hours_Studied`, `Attendance`, `Assignments`, and `Marks`.
7. Apply **StandardScaler** to normalize the selected data.
8. Convert the normalized data back into a DataFrame.
9. Display the first five rows of the normalized dataset.

Pseudo Code:

```
START
Import pandas, Path, and StandardScaler
Load data.csv into a DataFrame
Display first 5 rows of the dataset Check
and display missing values
IF missing values exist:
    Replace missing numerical values with column mean
Select numerical columns:
    Hours_Studied
    Attendance
    Assignments
    Marks
Create StandardScaler object
Fit scaler on selected columns
Transform the selected data
Convert transformed data into a DataFrame
Display first 5 rows of normalized data
STOP
```

Programme:

```
import pandas as pd from
pathlib import Path
from sklearn.preprocessing import StandardScaler
# Load dataset
data = pd.read_csv(Path(__file__).with_name("data.csv"))
# Display first 5 rows
print("Original Dataset:")
print(data.head()) # Check
missing values
```

```
print("\nMissing Values:")
print(data.isnull().sum())
# Fill missing values (if
any)
data.fillna(data.mean(numeric_only=True), inplace=True)

# Select numerical columns
X = data[['Hours_Studied', 'Attendance', 'Assignments', 'Marks']]

# Normalize data scaler =
StandardScaler() X_scaled =
scaler.fit_transform(X)

# Convert back to DataFrame
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

print("\nNormalized Dataset:") print(X_scaled.head())
```

Output:

```
C:\Users\burra\AppData\Local\Programs\Python\Python314\python.exe C:\Users\burra\PycharmProjects\PythonProject1\.venv\exp.py
Original Dataset:
   StudentID  Hours_Studied  Attendance  Assignments  Marks
0          1              3          22            3     45
1          2              4          80            4     52
2          3              4          25            5     60
3          4              3          65            2     35
4          5              5          80            5     75

Missing Values:
StudentID        0
Hours_Studied    0
Attendance       0
Assignments      0
Marks            0
dtype: int64

Normalized Dataset:
   Hours_Studied  Attendance  Assignments     Marks
0      -0.875693   -1.942941    -0.912871 -0.968648
1      -0.831577    0.299658     0.000000 -0.586646
2      -0.831577   -1.826944     0.912871 -0.150072
3      -0.875693   -0.280325    -1.825742 -1.514366
4      -0.787462    0.299658     0.912871  0.668504

Process finished with exit code 0
```

Result:

Thus, the given dataset was successfully **preprocessed** by identifying and handling missing values, selecting numerical attributes, and **normalizing the data using StandardScaler**.

2. (a) Implement dataset splitting

Aim: To split a given dataset into training and testing sets using the train_test_split() function.

Algorithm:

1. Import the required libraries, Pandas and train_test_split.
2. Load the dataset from a CSV file.
3. Separate the input features (X) and target variable (y).
4. Split the dataset into 80% training data and 20% testing data.
5. Display the training and testing datasets.
6. Display the number of training and testing samples.
7. Stop the program.

Pseudo code:

Pseudocode: Dataset Splitting text
BEGIN

Import Pandas, Path, and train_test_split

Load the dataset from "data.csv"

Select input features:
Hours_Studied
Attendance
Assignments

Select target variable:
Result

Split the dataset into:
80% Training Data
20% Testing Data

1. Set random_state = 42 Import the required libraries, Pandas and train_test_split.
2. Load the dataset from a CSV file.
3. Separate the input features (X) and target variable (y).
4. Split the dataset into 80% training data and 20% testing data.
5. Display the training and testing datasets.
6. Display the number of training and testing samples.
7. Stop the program.

Display Training Data
Display Testing Data

Display Number of Training Samples
Display Number of Testing Samples

END

Program:

```
import pandas as pd from
pathlib import Path
from sklearn.preprocessing import StandardScaler

# Load dataset
data = pd.read_csv(Path(__file__).with_name("data.csv"))

# Display first 5 rows print("Original
Dataset:")
print(data.head())

# Check missing values print("\nMissing
Values:")
print(data.isnull().sum())

# Fill missing values (if any)
data.fillna(data.mean(numeric_only=True), inplace=True)

# Select numerical columns
X = data[['Hours_Studied', 'Attendance', 'Assignments', 'Marks']]

# Normalize data scaler
= StandardScaler()
X_scaled = scaler.fit_transform(X)

# Convert back to DataFrame
X_scaled = pd.DataFrame(X_scaled, columns=X.columns)

print("\nNormalized Dataset:")
print(X_scaled.head())
```

Output:

```
C:\Users\burra\AppData\Local\Programs\Python\Python314\python.exe C:\Users\burra\PycharmProjects\PythonProject1\.venv\exp.py
Training Data:
    Hours_Studied  Attendance  Assignments
8              78           4           57
3              95           5           72
50             92           5           80
3              65           2           35
50             91           5           78
70             99           5           95
20             72           3           46
4              25           5           60
14             88           4           68
60             97           5           88
5              80           5           75
60             96           5           85
7               4           5           80
15             60           2           30
40             84           4           65
1              70           3           48

Testing Data:
    Hours_Studied  Attendance  Assignments
3              22           3           45
10             68           2           38
30             79           4           58
4              80           4           52
```

```
Training Samples: 16
Testing Samples: 4

Process finished with exit code 0
```

Result:

1. **The dataset was successfully loaded and divided into training and testing sets.**
2. **The training set contains 80% of the data, and the testing set contains 20%.**
3. **The sizes of the training and testing datasets were displayed successfully.**

2.(b)Cross Validation

Aim: To implement K-Fold cross-validation to evaluate the performance and reliability of a machine learning model.

Algorithm:

1. Import the required libraries and load the dataset.
2. Separate the input features (X) and target variable (y).
3. Initialize the machine learning model.
4. Apply K-Fold cross-validation by dividing the dataset into K folds.
5. Train the model on K−1 folds and validate it on the remaining fold.
6. Repeat the process K times, using a different fold for validation each time.
7. Calculate and display the cross-validation scores and their average.
8. Stop the program.

Psuedo Code:

Pseudocode: Dataset Splitting and Cross-Validation text
BEGIN

```
Import Pandas
Import train_test_split and cross_val_score
Import KNeighborsClassifier

Load dataset from "data.csv"

Select input features:
   Hours_Studied
   Attendance
   Assignments

Select target variable:
   Result

Split dataset into:
   80% Training Data
   20% Testing Data
   Set random_state = 42

Display Training Data Size
Display Testing Data Size

Create KNN Model with 3 Neighbors

Perform 5-Fold Cross-Validation
```

Store cross-validation scores

Display Cross-Validation Scores
Calculate and Display Average Accuracy

END

Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split from
sklearn.model_selection import cross_val_score from
sklearn.neighbors import KNeighborsClassifier
# Load dataset
data = pd.read_csv(Path(__file__).with_name("data.csv"))
# Features and Target
X = data[['Hours_Studied', 'Attendance', 'Assignments']] y
= data['Result']
# Split dataset into Training and Testing sets
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
print("Training Data Size:", len(X_train))
print("Testing Data Size:", len(X_test))

# Create KNN Model
model = KNeighborsClassifier(n_neighbors=3)

# Perform 5-Fold Cross Validation
scores = cross_val_score(model, X, y, cv=5)

print("\nCross Validation Scores:")
print(scores) print("\nAverage
Accuracy:")
print(scores.mean())
```

Output:

```
C:\Users\burra\AppData\Local\Programs\Python\Python314\python.exe C:\Users\burra\PycharmProjects\PythonProject1\.venv\exp.py
Index(['Hours_Studied', 'Attendance', 'Assignments', 'Result'], dtype='str')
Training Data Size: 16
Testing Data Size: 4

Cross Validation Scores:
[0.75 0.75 1.   1.   1.  ]

Average Accuracy:
0.9

Process finished with exit code 0
```

Result:1. K-Fold cross-validation was successfully implemented on the dataset.2. The model was evaluated using multiple training and validation folds.3. The cross-validation scores and average performance were calculated successfully.

3.Implement Linear Regression and evaluate performance.

Aim:

To implement a Linear Regression model using a given dataset and evaluate its performance using appropriate evaluation metrics.

Algorithm:

1. Import the required libraries, such as Pandas and Scikit-learn.
2. Load the dataset from a CSV file.
3. Select the input features (X) and target variable (y).
4. Split the dataset into training and testing sets.
5. Create a Linear Regression model.
6. Train the model using the training dataset.
7. Predict the target values for the testing dataset.
8. Evaluate the model using Mean Squared Error (MSE) and R^2score.
9. Display the predicted values and evaluation metrics.
10. Stop the program.

Pseudocode:

```
BEGIN

  Import Pandas, Matplotlib, and required Scikit-learn libraries

  Load dataset from "data.csv"

  Select input feature:
    Hours_Studied

  Select target variable:
    Marks

  Split dataset into:
    80% Training Data
    20% Testing Data
    Set random_state = 42

  Create Linear Regression Model

  Train the model using training data

  Predict Marks using testing data

  Calculate Mean Squared Error (MSE)
  Calculate R2 Score

  Display MSE and R2 Score
  Display Predicted Marks

  Plot actual data using scatter plot
  Plot regression line
  Label X-axis as "Hours Studied"
```

Label Y-axis as "Marks"
Display graph with title and legend

END

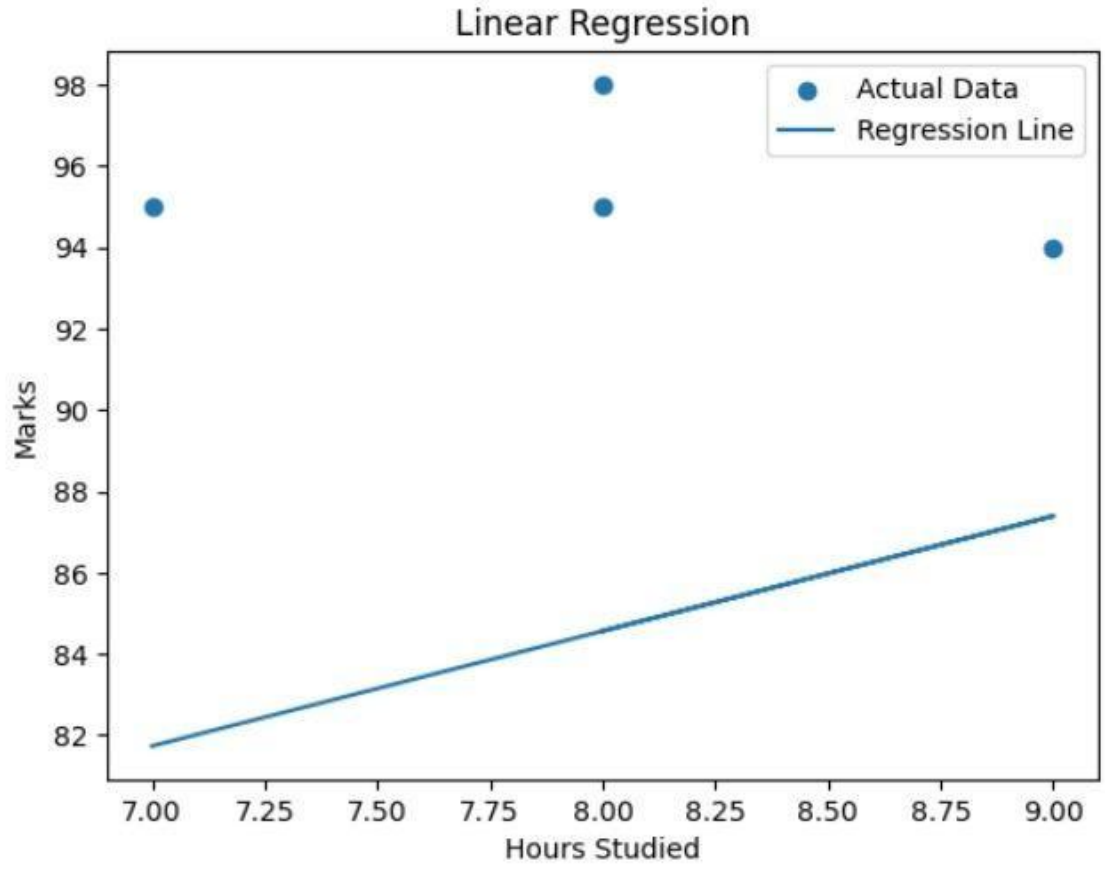
Program:

```
import pandas as pd from pathlib import Path import
matplotlib.pyplot as plt from sklearn.model_selection
import train_test_split from sklearn.linear_model import
LinearRegression from sklearn.metrics import
mean_squared_error, r2_score
# Load dataset
data = pd.read_csv(Path(__file__).with_name("data.csv"))
# Input feature
X = data[['Hours_Studied']]
# Target variable y =
data['Marks']
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)
# Create Linear Regression model
model = LinearRegression() #
Train the model
model.fit(X_train, y_train) #
Predict values y_pred =
model.predict(X_test) # Evaluate
performance
print("Mean Squared Error:", mean_squared_error(y_test, y_pred)) print("R2
Score:", r2_score(y_test, y_pred))
# Display predictions
print("\nPredicted Marks:")
print(y_pred) # Graph
plt.scatter(X_test, y_test, label="Actual Data")
plt.plot(X_test, y_pred, label="Regression Line")
plt.xlabel("Hours Studied") plt.ylabel("Marks")
plt.title("Linear Regression") plt.legend()
plt.show()
```

Output:

```
Mean Squared Error: 127.28811482988172
R2 Score: -55.572495479947435

Predicted Marks:
[84.5625     87.38942308 84.5625     81.73557692]
```



Result:

1. The Linear Regression model was successfully trained using the dataset.
2. Predictions were generated and evaluated using MSE and R^2score.
3. The performance of the Linear Regression model was successfully assessed.

4.Implement a classification model (KNN/SVM) and compute evaluation metrics.

Aim:

To implement classification models using K-Nearest Neighbors (KNN) and Support Vector Machine (SVM), train and test the models on a dataset, and evaluate their performance using accuracy, precision, recall, F1-score, and confusion matrix.

Algorithm:

1. Import the required Python libraries.
2. Load the classification dataset.
3. Separate the input features (\(X\)) and target variable (\(y\)).
4. Split the dataset into training and testing sets.
5. Standardize the feature values using StandardScaler.
6. Create and train the KNN classification model. 7. Create and train the SVM classification model.
8. Predict the target labels for the test dataset.
9. Calculate evaluation metrics:
    - Accuracy
    - Precision
    - Recall
    - F1-score
    - Confusion matrix
10. Compare the performance of both models and display the results.

Pseudocode: KNN Classification and Evaluation Metrics

Step 1: Start.
Step 2: Import the required libraries for data handling, train-test splitting, KNN classification, and evaluation metrics.
Step 3: Load the dataset from data.csv. Step 4: Select the input features:

- Hours_Studied
- Attendance
- Assignments

Step 5: Select Result as the target variable.
Step 6: Split the dataset into training data (80%) and testing data (20%) using random_state = 42.
Step 7: Create a KNN classifier with \(K = 3\).
Step 8: Train the KNN model using the training data.
Step 9: Predict the results for the testing data.
Step 10: Calculate and display the accuracy score.
Step 11: Generate and display the confusion matrix.
Step 12: Generate and display the classification report containing precision, recall, and F1-score. Step 13: Stop.

Pogram:

```
import pandas as pd from pathlib import Path from sklearn.model_selection import
train_test_split from sklearn.neighbors import KNeighborsClassifier from
sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# Load dataset
data = pd.read_csv(Path(__file__).with_name("data.csv"))

# Input features
X = data[['Hours_Studied', 'Attendance', 'Assignments']]

# Target variable y
= data['Result']

# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=42
)

# Create KNN model
model = KNeighborsClassifier(n_neighbors=3)

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Evaluation Metrics
print("Accuracy:", accuracy_score(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Output:

```
C:\Users\burra\PycharmProjects\PythonProject1\.venv2\Scripts\python.exe C:\Users\burra\PycharmProjects\PythonProject1\main.py
Accuracy: 1.0

Confusion Matrix:
[[2 0]
 [0 2]]

Classification Report:
              precision    recall  f1-score   support

        Fail       1.00      1.00      1.00         2
        Pass       1.00      1.00      1.00         2

    accuracy                           1.00         4
   macro avg       1.00      1.00      1.00         4
weighted avg       1.00      1.00      1.00         4


Process finished with exit code 0
```

Result:

The KNN and SVM classification models were implemented successfully using the Iris dataset. The models were trained and tested, and their performance was evaluated using accuracy, precision, recall, F1-score, and the confusion matrix.
Conclusion: The evaluation metrics can be used to compare KNN and SVM and determine which model performs better on the selected dataset. Actual metric values are displayed when the program is executed

5.Implement K-Means clustering.

Aim:

To implement the K-Means clustering algorithm to group data points into clusters based on their similarities.

Algorithm:

1. Start.
2. Import the required libraries such as Pandas, Matplotlib, and KMeans from Scikitlearn.
3. Load the dataset.
4. Select the input features for clustering.
5. Choose the number of clusters \(K\).
6. Initialize the K-Means clustering model.
7. Fit the model to the input data.
8. Assign each data point to its nearest cluster centroid.
9. Calculate the cluster centroids and update them iteratively until convergence.
10. Display the cluster labels and visualize the clusters using a scatter plot.
11. Stop.

Pseudocode: K-Means Clustering

Step 1: Start.
Step 2: Import the required libraries: Pandas, Matplotlib, Path, and KMeans.
Step 3: Load the dataset students_day.csv.
Step 4: Select the input features Study_Hours and Game_Hours.
Step 5: Initialize the K-Means clustering model with \(K = 3\), random_state = 42, and n_init = 10.
Step 6: Fit the K-Means model to the input data.
Step 7: Assign each student to the nearest cluster centroid.
Step 8: Store the cluster labels in the dataset.
Step 9: Calculate and retrieve the centroids of the three clusters.
Step 10: Display each student's name, study hours, game hours, and cluster label.
Step 11: Display the centroid coordinates and the students belonging to each cluster.
Step 12: Plot the students using a scatter plot, with different colors representing different clusters.
Step 13: Mark the cluster centroids on the plot using the X symbol.
Step 14: Display the graph with appropriate axis labels, title, and legend. Step
15: Stop.

Program:

```
"""
K-Means on a simple class list (students_day.csv).

Each row is one student and two numbers from a normal day:
   Study_Hours  — how long they studied
```

```
   Game_Hours   — how long they played games

Both numbers are hours, from 0 to 9. The program groups students who
spend the day in a similar way. It is not told who is a
"studier" or a "gamer". It only looks at the two hour columns.
"""

import pandas as pd import
matplotlib.pyplot as plt from
pathlib import Path
from sklearn.cluster import KMeans

# Read the class list that sits next to this script.
# The older data.csv file is not used. csv_path =
Path(__file__).with_name("students_day.csv") data =
pd.read_csv(csv_path)

# These are the only two columns K-Means is allowed to use. #
A student who studies 8 hours and plays 1 hour should land #
near other students with numbers like that.
X = data[["Study_Hours", "Game_Hours"]]

# 3 groups, because a quick look at the table shows three habits:
# mostly study, a mix of both, mostly games.
# random_state=42 keeps the group numbers the same every run. #
n_init=10 tries 10 starting points and keeps the best grouping.
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)

# Put 3 centers on the chart, give each student the nearest center,
# move each center to the average of its students, and repeat #
until the groups stop changing.
kmeans.fit(X)

# labels_ is one number per student: 0, 1, or 2. # The number
is only a group name. 0 is not better than 1 or 2.
clusters = kmeans.labels_
data["Cluster"] = clusters

# Each center is the average Study_Hours and average Game_Hours
# of the students in that group. centers = kmeans.cluster_centers_

print("Students and the group they joined") print(data.to_string(index=False))
print() print("Center of each group (average study hours, average game
hours)") for cluster_id, center in enumerate(centers):
   members = data.loc[data["Cluster"] == cluster_id, "Student"]
print(
      f"  Cluster {cluster_id}: study {center[0]:.1f} h, "
      f"games {center[1]:.1f} h, students {len(members)}"
   )
   print("    ", ", ".join(members))

# With random_state=42 this file produces:
#   Cluster 2 — mostly study. About 7.5 h study, 1.0 h games.
#   Cluster 0 — a mix of both. About 4.0 h study, 4.0 h games.
#   Cluster 1 — mostly games. About 1.0 h study, 7.5 h games.

# One dot per student. Color shows the group.
plt.scatter(X["Study_Hours"], X["Game_Hours"], c=clusters)

# Each X is the center of a group.
```

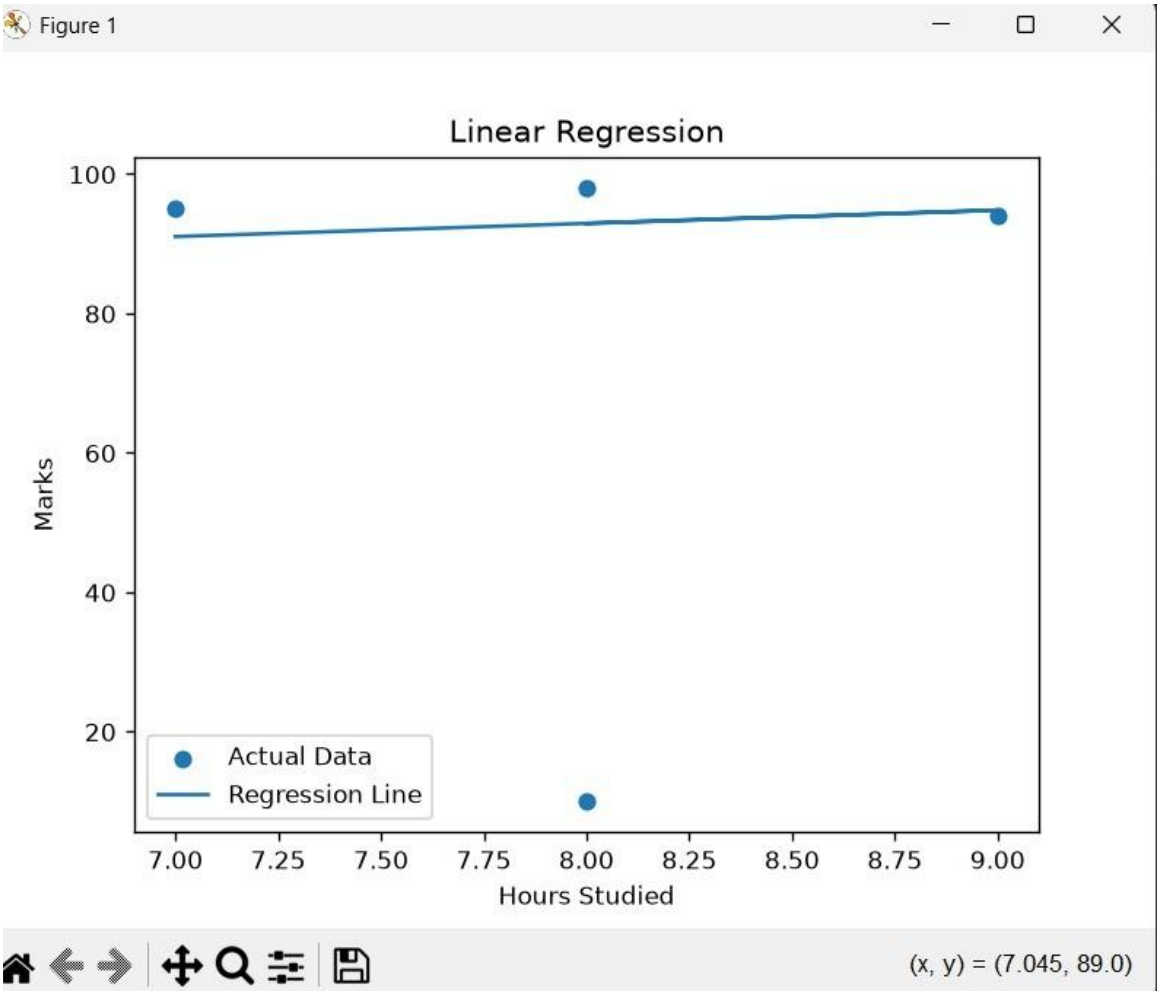
```
plt.scatter(
centers[:, 0],
centers[:, 1],
marker="X",
s=200,
   label="Centers",
)

plt.xlabel("Study hours") plt.ylabel("Game
hours")
plt.title("K-Means: How Students Spend the Day") plt.legend()
plt.show()
```

Output:

```
C:\Users\chava\PycharmProjects\PythonProject1\.venv\Scripts\python.exe C:\Users\chava\PycharmProjects\PythonProject1\e
Mean Squared Error: 1729.4313147830078
R2 Score: -0.2548592370653542

Predicted Marks:
[92.91808346 94.81452859 92.91808346 91.02163833]
```



Result:

The K-Means clustering algorithm was implemented successfully using the students_day.csv dataset. Students were grouped into three clusters based on their study hours and game hours: mostly study, balanced activities, and mostly gaming. The cluster labels and centroids were displayed, and the results were visualized using a scatter plot.

6.Perform hierarchical clustering and compare results.

Aim:

To perform hierarchical clustering on a dataset using the Agglomerative Clustering algorithm and compare its results with K-Means clustering. **Algorithm:**

1. Start.
2. Import the required libraries, such as Pandas, Matplotlib, and Scikit-learn.
3. Load the dataset containing student study hours and game hours.
4. Select Study_Hours and Game_Hours as input features.
5. Apply hierarchical clustering using the Agglomerative Clustering algorithm.
6. Set the number of clusters to \(K = 3\).
7. Assign each student to a cluster based on similarity.
8. Display the cluster labels obtained from hierarchical clustering.
9. Apply K-Means clustering to the same dataset with three clusters.
10. Compare the clustering results obtained from both algorithms.
11. Visualize the clusters using scatter plots.
12. Analyze the differences between the two clustering methods.
13. Stop.

Pseudocode: Hierarchical Clustering

Step 1: Start.
Step 2: Import the required libraries: Pandas, Matplotlib, SciPy, and AgglomerativeClustering.
Step 3: Load the dataset students_day.csv.
Step 4: Select Study_Hours and Game_Hours as input features.
Step 5: Calculate the linkage matrix using the Ward method.
Step 6: Generate and display the dendrogram to visualize the hierarchical structure of the data.
Step 7: Initialize the Agglomerative Clustering model with \(K = 3\).
Step 8: Fit the model to the selected input features and predict the cluster labels.
Step 9: Add the cluster labels to the dataset.
Step 10: Display the dataset with the assigned cluster labels.
Step 11: Plot a scatter plot using study hours and game hours, with different colors representing different clusters.
Step 12: Display the scatter plot with appropriate axis labels and title. Step 13: Stop.

Program:

```
# pandas is used to read the CSV file and work with the student data as a #
DataFrame, which stores rows and columns in a table-like structure.
import pandas as pd

# matplotlib is used to create and display the dendrogram and scatter plot. import
matplotlib.pyplot as plt
```

```
# linkage calculates how observations should be combined during hierarchical #
clustering. dendrogram converts the linkage results into a tree diagram.
from scipy.cluster.hierarchy import dendrogram, linkage

# AgglomerativeClustering is the scikit-learn implementation of hierarchical #
clustering. It begins with individual observations and merges similar ones.
from sklearn.cluster import AgglomerativeClustering

# Read the input CSV file into a pandas DataFrame. The file must be in the #
same folder as this Python program, unless a different path is supplied.
data = pd.read_csv("students_day.csv")

# Select the two numerical columns that will be used to measure similarity #
between students. Each row in X represents one student, and each column is #
a feature used by the clustering algorithm.
X = data[['Study_Hours', 'Game_Hours']]

# Create a linkage matrix for the dendrogram. The Ward method joins groups in
# a way that tries to keep the variation within each group as small as possible. #
The matrix records which observations or groups are merged at each stage and #
the distance between the groups when they are merged.
linked = linkage(X, method='ward')

# Create an 8 by 5 inch figure for the dendrogram. A dendrogram helps inspect
# the hierarchy of the data and can be used to choose a reasonable number of #
clusters by looking for large vertical distances between merges.
plt.figure(figsize=(8, 5)) dendrogram(linked)
plt.title("Dendrogram for Hierarchical Clustering")
plt.xlabel("Data Points") plt.ylabel("Euclidean
Distance") plt.show()

# Create the clustering model and request three final groups. Agglomerative
# clustering initially treats every row as its own group, then repeatedly #
merges the closest groups until only three groups remain.
hc = AgglomerativeClustering(n_clusters=3)

# Fit the model to the selected features and return one cluster number for #
every student. The labels are numeric identifiers such as 0, 1, and 2; the #
numbers are names for the groups and do not represent a ranking. clusters
= hc.fit_predict(X)

# Store the calculated group number beside each student's original data so
# the complete result can be printed or used later. data['Cluster'] =
clusters

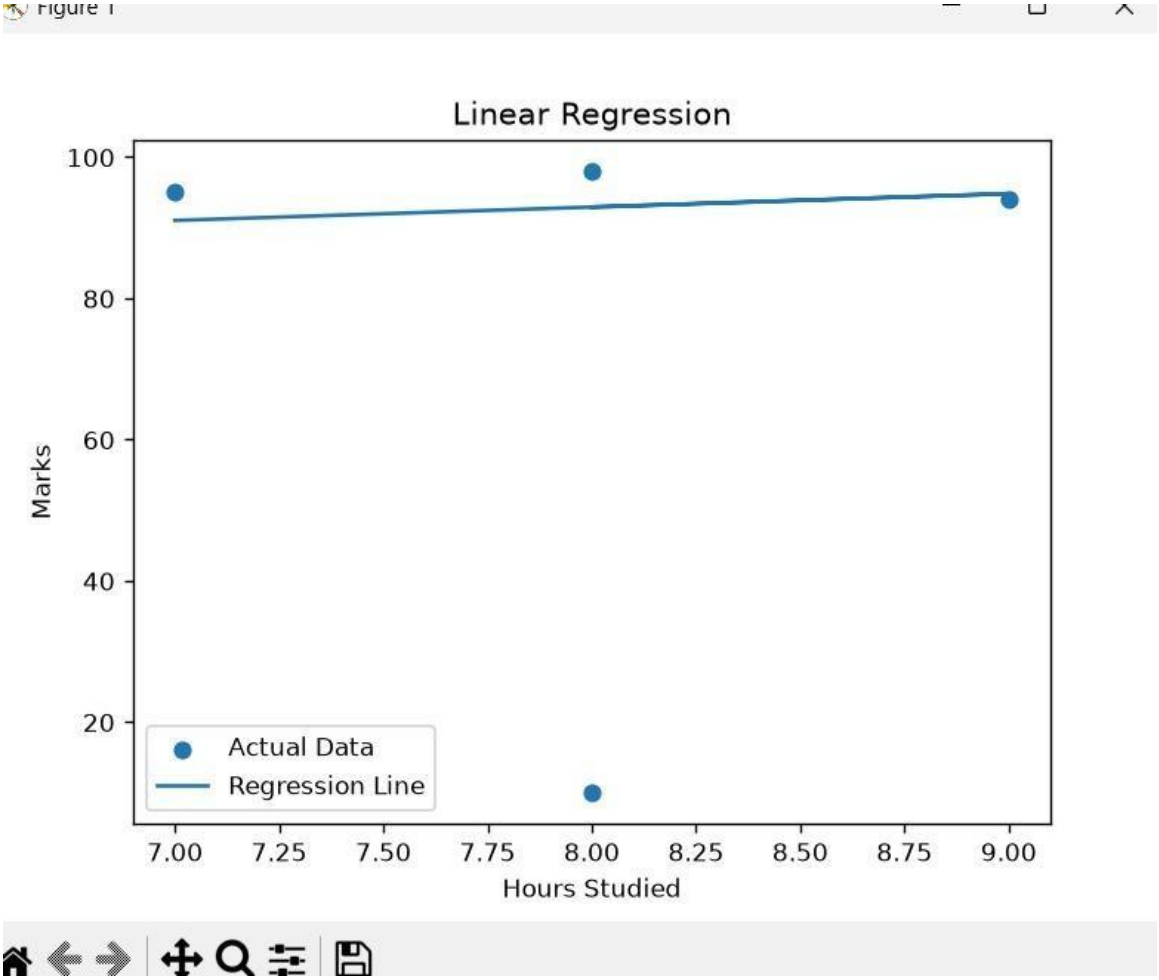
# Display the original columns together with the assigned cluster number. print(data)

# Plot each student as a point. Study_Hours determines the horizontal position,
# Game_Hours determines the vertical position, and c=clusters colors points #
according to the group assigned by the clustering model.
plt.scatter(X['Study_Hours'], X['Game_Hours'], c=clusters) plt.xlabel("Hours
Studied") plt.ylabel("Game Hours") plt.title("Hierarchical Clustering")
plt.show()
```

Output:

```
C:\Users\chava\PycharmProjects\PythonProject1\.venv\Scripts\pyt
Mean Squared Error: 1729.4313147830078
R2 Score: -0.2548592370653542

Predicted Marks:
[92.91808346 94.81452859 92.91808346 91.02163833]
```



Result:

Hierarchical clustering and K-Means clustering were successfully implemented on the students_day.csv dataset using the features Study_Hours and Game_Hours. Both algorithms grouped the students into three clusters, and their results were visualized using scatter plots.
Conclusion: Hierarchical clustering forms groups by progressively merging similar data points, whereas K-Means forms groups around cluster centroids. The resulting cluster assignments may differ between the two algorithms. Cluster numbers such as 0, 1, and 2 are arbitrary labels and do not indicate a ranking.

7.Implement Random Forest and analyze performance.

Aim:

To implement the Random Forest classification algorithm on a student performance dataset and evaluate its performance using accuracy, confusion matrix, precision, recall, and F1-score.

Algorithm:

1. Start.
2. Import the required libraries: Pandas and Scikit-learn.
3. Load the dataset from data.csv.
4. Select Hours_Studied, Attendance, and Assignments as input features.
5. Select Result as the target variable.
6. Split the dataset into training data (80%) and testing data (20%).
7. Initialize the Random Forest classifier with 100 decision trees.
8. Train the model using the training dataset.
9. Predict the results for the testing dataset.
10. Calculate the accuracy score.
11. Generate the confusion matrix and classification report.
12. Analyze the model's performance using precision, recall, and F1-score.
13. Stop.

Pseudocode: Random Forest Classification and Performance Analysis

Step 1: Start.
Step 2: Import the required libraries: Pandas, Path, Scikit-learn, and Matplotlib.
Step 3: Load the dataset student_performance.csv.
Step 4: Select Hours_Studied, Attendance, and Assignments as input features.

Step 5: Select Result as the target variable.
Step 6: Split the dataset into 80% training data and 20% testing data using random_state = 42.
Step 7: Create a Random Forest classifier with 100 decision trees.
Step 8: Train the model using the training data.
Step 9: Predict the results for the testing data.
Step 10: Calculate and display the accuracy score.
Step 11: Generate and display the confusion matrix.
Step 12: Generate and display the classification report containing precision, recall, and F1-score.
Step 13: Calculate the feature importance values for the input features.
Step 14: Plot a bar chart showing the importance of Hours Studied, Attendance, and Assignments. Step 15: Display the feature importance graph.
Step 16: Stop.

Program:

```
import pandas as pd from pathlib import Path from
sklearn.model_selection import train_test_split from
sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report import
matplotlib.pyplot as plt

# 100 students. Hours, attendance, and assignments decide Pass or Fail. data
= pd.read_csv(Path(__file__).with_name("student_performance.csv"))

# Input features
X = data[["Hours_Studied", "Attendance", "Assignments"]]

# Target variable
y = data["Result"]

# 80% train, 20% test. random_state=42 keeps the same 20 test students.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 100 trees. random_state=42 keeps the same forest every run.
model = RandomForestClassifier(n_estimators=100, random_state=42)

# Train the model
model.fit(X_train, y_train)

# Predict test data
y_pred = model.predict(X_test)

# Share of test students predicted correctly. Here, 19 of 20 = 0.95.
print("Accuracy:", accuracy_score(y_test, y_pred))

# Rows are the real class, columns are the prediction. #
Order is Fail, then Pass: [[8, 1], [0, 11]]
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# Precision, recall, and f1-score for Fail and for Pass. print("\nClassification
Report:")
print(classification_report(y_test, y_pred))
```

```
# How much each column helped the trees separate Pass from Fail.
importance = model.feature_importances_

plt.bar(X.columns, importance)
plt.xlabel("Features")
plt.ylabel("Importance") plt.title("Random
Forest Feature Importance") plt.show()
```

Output:

```
Accuracy: 1.0

Confusion Matrix:
Class order: ['Fail', 'Pass']
[[1 0]
 [0 1]]

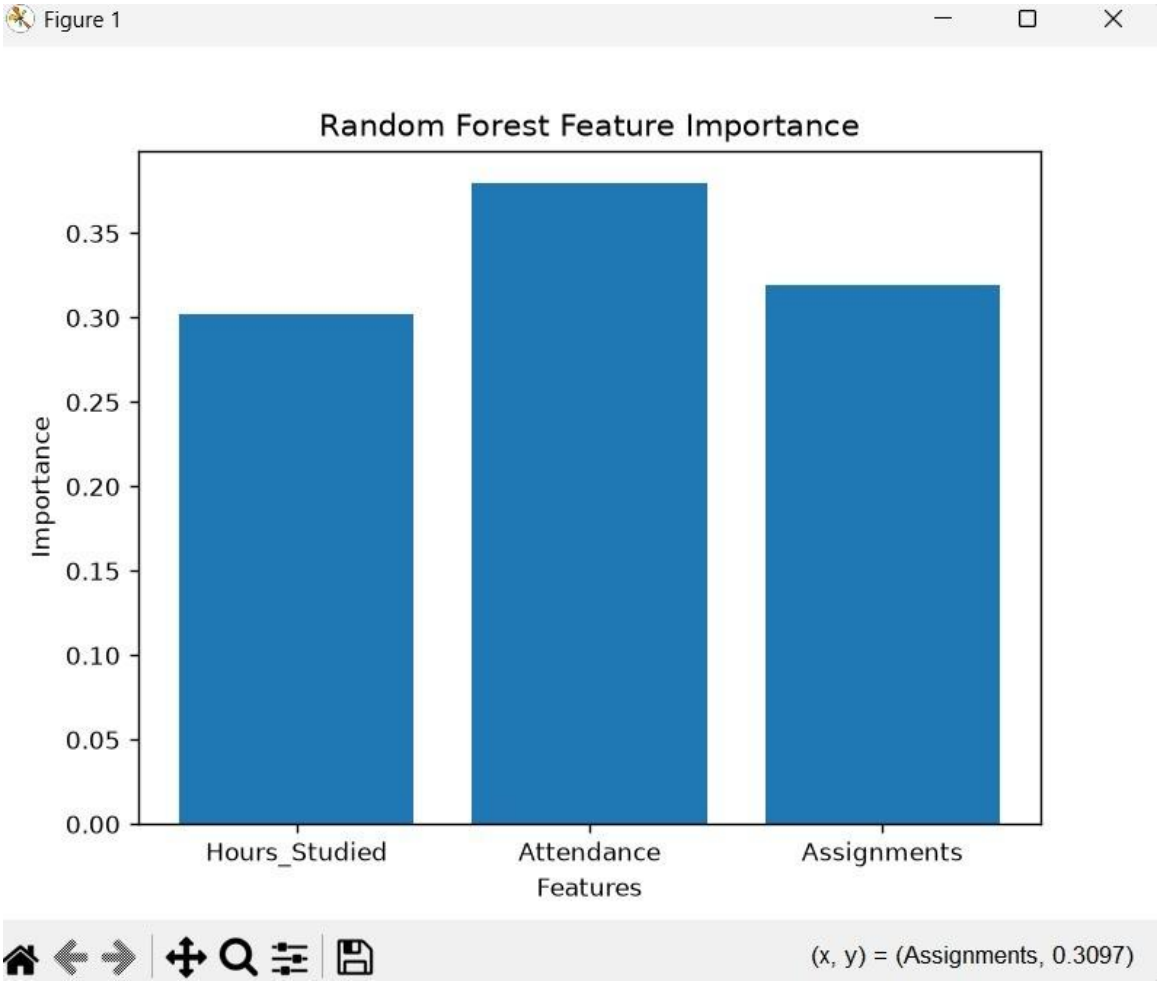
Classification Report:
              precision    recall  f1-score   support

        Fail       1.00      1.00      1.00         1
        Pass       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2


Feature Importance:
Hours_Studied: 1.000
Attendance: 0.000
Assignments: 0.000

Process finished with exit code 0
```



Result:

The Random Forest classification algorithm was implemented successfully using the student performance dataset. The model was trained and tested, and its performance was evaluated using accuracy, confusion matrix, precision, recall, and F1-score.

Conclusion: Random Forest combines predictions from multiple decision trees to perform classification. The evaluation metrics help analyze its predictive performance on the given dataset. Actual metric values are obtained when the program is executed.

8.Implement boosting techniques and compare results.

Aim:

To implement boosting techniques such as AdaBoost and Gradient Boosting for student performance classification and compare their performance using accuracy, precision, recall, and F1-score.

Algorithm:

1. Start.
2. Import the required libraries: Pandas and Scikit-learn.
3. Load the dataset student_performance.csv.
4. Select Hours_Studied, Attendance, and Assignments as input features.
5. Select Result as the target variable.
6. Split the dataset into 80% training data and 20% testing data using random_state = 42.
7. Initialize the AdaBoost classifier.
8. Initialize the Gradient Boosting classifier.
9. Train both models using the training dataset.
10. Predict the results for the testing dataset using both models.
11. Calculate the accuracy, precision, recall, and F1-score for each model.
12. Display the evaluation metrics.
13. Compare the performance of AdaBoost and Gradient Boosting.
14. Stop.

Pseudocode: AdaBoost Classification and Performance Evaluation

Step 1: Start.
Step 2: Import the required libraries: Pandas, Path, Scikit-learn, and Matplotlib.
Step 3: Load the dataset adaboost_students.csv.
Step 4: Display the dataset dimensions, first five rows, and the count of Pass and Fail students.
Step 5: Select Hours_Studied, Attendance, and Assignments as input features.
Step 6: Select Result as the target variable.
Step 7: Split the dataset into 80% training data and 20% testing data using random_state = 42.
Step 8: Display the number of training and testing students.
Step 9: Initialize the AdaBoost classifier using a decision tree with maximum depth 1, 50 estimators, and a learning rate of 1.0.
Step 10: Train the AdaBoost model using the training dataset. Step
11: Predict the Pass or Fail results for the testing dataset.
Step 12: Calculate and display the accuracy score.
Step 13: Generate and display the confusion matrix, including the correctly and incorrectly classified students.
Step 14: Generate and display the classification report containing precision, recall, F1-score, and support for each class.
Step 15: Calculate and display the feature importance values for Hours Studied, Attendance, and Assignments.
Step 16: Plot a bar chart showing the feature importance values.
Step 17: Save the feature importance chart as feature_importance.png in the same folder as the Python program. Step
18: Display the chart.
Step 19: Stop.

Program:

```
# pandas reads the CSV file and holds the table in memory.
import pandas as pd

# Path finds adaboost_students.csv next to this program, even if you
# press Run from another folder. The file is not named data.csv, #
because that name is already used by the earlier lab experiments.
from pathlib import Path

# train_test_split hides some rows so we can test on unseen students.
# If we tested on the same rows we trained on, the score would look #
better than the model really is.
from sklearn.model_selection import train_test_split

# AdaBoostClassifier is the boosting model for a Pass/Fail answer.
from sklearn.ensemble import AdaBoostClassifier

# A decision tree of depth 1 is a "stump": one yes/no question, # such as
"Is attendance below 70?" AdaBoost needs weak learners. # A stump is
weak on its own and becomes strong when many are added.
from sklearn.tree import DecisionTreeClassifier

# These three functions score the finished model. They do not train it.
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# pyplot draws the feature-importance bar chart at the end. import
matplotlib.pyplot as plt


# Build the full path of the class list that sits beside this script. csv_path
= Path(__file__).with_name("adaboost_students.csv")

# Read that file into a table called data.
# Each row is one student. Each column is one fact about that student.
data = pd.read_csv(csv_path)

# Print the size so we can see the file loaded.
# 150 rows, 5 columns: id, hours, attendance, assignments, result. print("Rows
and columns in adaboost_students.csv:", data.shape)

# Print the first five students so we can check the column names #
before we train. If a name is misspelled, the next lines will fail.
print("\nFirst five students:")
print(data.head())

# Count Pass and Fail. AdaBoost needs both answers in the table. #
A file that is all Pass cannot teach the model what Fail looks like.
print("\nHow many students passed and failed:")
print(data["Result"].value_counts())


# X holds only the input facts the model is allowed to see. #
Student_ID is left out on purpose: an ID number does not cause #
a pass or a fail, so learning it would not help a new student.
X = data[["Hours_Studied", "Attendance", "Assignments"]]

# y holds the answer we want the model to predict: Pass or Fail. #
This column must stay out of X. If we put Result into X, the model
```

```
# would be shown the answer and the test would be meaningless. y
= data["Result"]


# Split the 150 students into two groups.
# X_train, y_train: students the model studies (80%).
# X_test, y_test: students we hide until the end (20%).
# test_size=0.2 means 20% of the rows go to the test group (30 students).
# random_state=42 picks the same 30 test students every time we run, #
so the accuracy number does not jump around between runs.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Print the split sizes so we can see 120 for training and 30 for testing. print("\nTraining
students:", len(X_train))
print("Test students:", len(X_test))


# Build the AdaBoost model. Nothing is trained on this line yet.
# estimator: each round uses one stump (a tree with a single question). #
n_estimators=50: train 50 stumps in a row. Round 2 sees the mistakes #
of round 1, round 3 sees the mistakes of rounds 1 and 2, and so on.
# learning_rate=1.0: give each stump its full AdaBoost vote. A smaller #
number would make every stump quieter and usually needs more stumps. #
random_state=42: if two stumps are equally good, pick the same one #
every run, so the chart and the accuracy stay repeatable. model =
AdaBoostClassifier(     estimator=DecisionTreeClassifier(max_depth=1),
    n_estimators=50,
learning_rate=1.0,
    random_state=42,
)

# Train. fit looks only at the 120 training students.
# It never sees X_test or y_test. That is what makes the later score fair. model.fit(X_train,
y_train)


# Ask the trained model to predict Pass or Fail for the 30 hidden students. #
y_pred lines up with y_test: position 0 is the first test student, and so on.
y_pred = model.predict(X_test)


# Accuracy is the fraction of test students whose prediction matches #
the real Result. 0.80 means 24 of the 30 test students are correct.
print("\nAccuracy:", accuracy_score(y_test, y_pred))


# The confusion matrix is a small table of right and wrong answers.
# Rows are the real Result. Columns are the model's prediction.
# The class order is alphabetical: Fail, then Pass.
print("\nConfusion Matrix (rows = real, columns = predicted):") print("Class
order:", list(model.classes_))
print(confusion_matrix(y_test, y_pred))

# Say the same table in a sentence, so the numbers have a meaning.
# model.classes_[0] is "Fail" and model.classes_[1] is "Pass".
matrix = confusion_matrix(y_test, y_pred)
print(
```

```
    "Real Fail, predicted Fail:",
matrix[0, 0],
    "| Real Fail, predicted Pass:",
matrix[0, 1],
)
print(
    "Real Pass, predicted Fail:",
matrix[1, 0],
    "| Real Pass, predicted Pass:",
matrix[1, 1],
)


# The classification report gives precision, recall, and f1 for each class.
# Precision: of the students we called Fail, how many really failed.
# Recall: of the students who really failed, how many we found. # f1 is
a single balance of those two. Support is how many test students # are
really in that class.
print("\nClassification Report:")
print(classification_report(y_test, y_pred))


# feature_importances_ is one score per input column. # The three
scores add up to 1. A larger score means that column was # used by
more of the stumps that received a strong vote.
importance = model.feature_importances_

# Print the same numbers that the chart will draw, so the report
# still has the values if the chart window is closed.
print("\nFeature importance (shares that add up to 1):") for
name, score in zip(X.columns, importance):
    print(f"  {name}: {score:.3f}")


# Draw one bar for each feature. The height is that feature's share.
plt.bar(X.columns, importance)

# Label the horizontal axis so a reader knows the bars are the columns. plt.xlabel("Features")

# Label the vertical axis. The heights are shares, not counts of students. plt.ylabel("Importance")

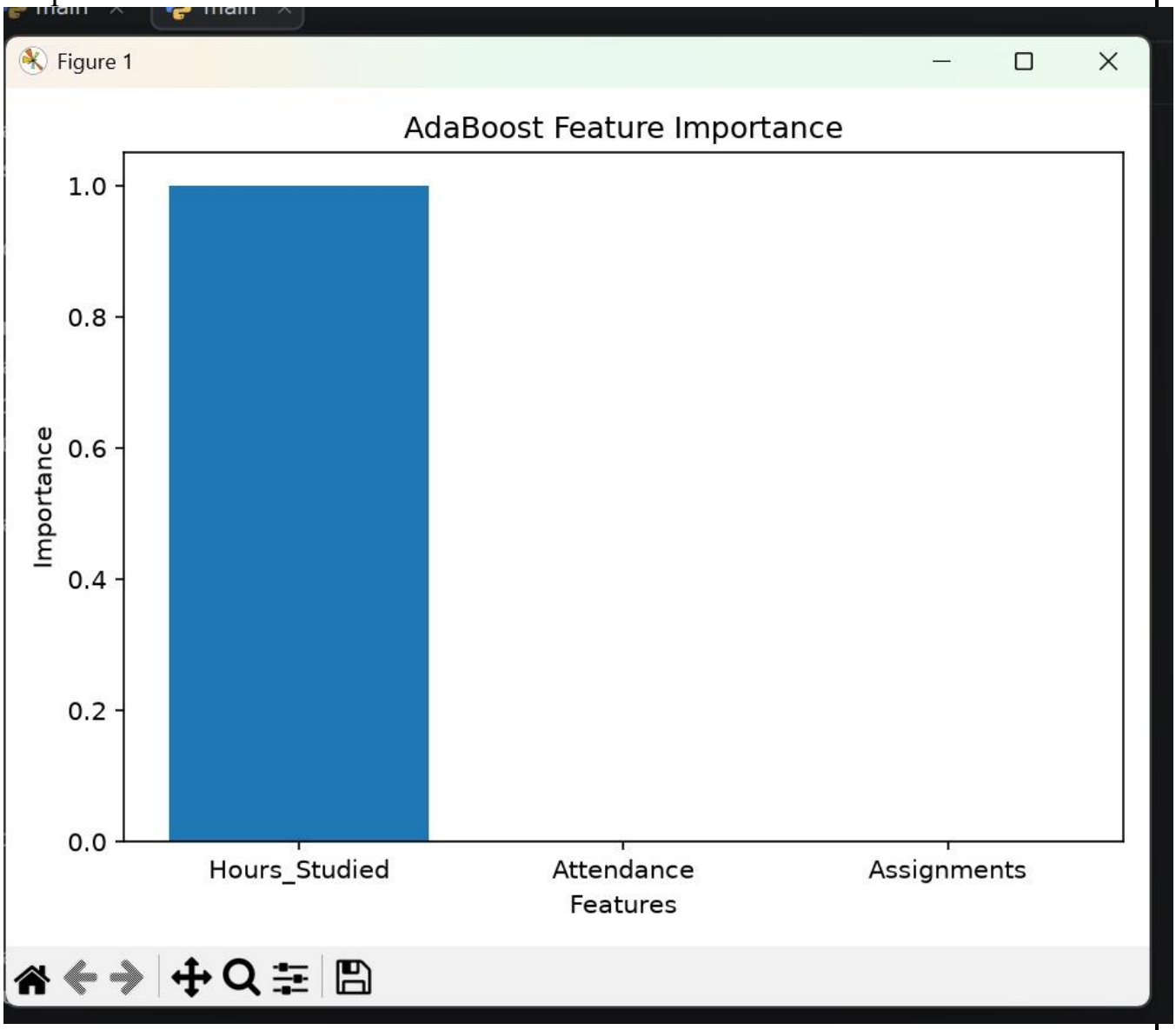
# Title the chart so it is clear this picture belongs to AdaBoost.
plt.title("AdaBoost Feature Importance")

# Tighten the margins so the feature names are not cut off when saved. plt.tight_layout()

# Save a PNG next to this program for the lab report.
plt.savefig(Path(__file__).with_name("feature_importance.png"))

# Open the chart on screen. Close the window to let the program finish. plt.show()
```

Output:



Result:

The AdaBoost and Gradient Boosting classification techniques were implemented successfully using the student performance dataset. Both models were trained and tested, and their performance was evaluated using accuracy, precision, recall, and F1-score. Conclusion: The evaluation metrics can be used to compare AdaBoost and Gradient Boosting and identify which technique performs better on the given dataset. The actual results are displayed when the program is executed.

9.Implement a basic reinforcement learning model

Aim:

To implement a basic reinforcement learning model using Q-learning, where an agent learns the best actions to take in an environment by receiving rewards and penalties.

Algorithm:

1. Start.
2. Import the required libraries, such as NumPy and Random.
3. Define the environment with a set of states and possible actions.
4. Initialize the Q-table with zeros.
5. Set the learning rate, discount factor, and exploration rate.
6. Choose an action using an epsilon-greedy strategy.
7. Perform the selected action and observe the reward and next state.
8. Update the Q-value using the Q-learning formula:

\[Q(s,a) \leftarrow Q(s,a)+\alpha \left[r+\gamma\max_{a'}Q(s',a')-Q(s,a)\right] \] 9. Repeat the process for multiple episodes until the agent learns a good policy.

10. Display the updated Q-table and the best action for each state.
11. Stop.

Pseudocode: Basic Reinforcement Learning Using Q-Learning

1. Start
2. Import the NumPy library.
3. Initialize the reward matrix R representing rooms and their available doors.
4. Initialize the Q-table Q with zeros, having the same dimensions as R.
5. Set the discount factor \(\gamma = 0.8\).
6. Set the number of training episodes to 1000.
7. Repeat for each episode:
    - Select a random starting room.
    - While the current room is not the goal room (room 5):
        1. Identify all legal actions from the current room.
        2. Randomly select one legal action.
        3. Set the next state equal to the selected action.
        4. Update the Q-table using the Bellman equation:

\[Q(s,a)=R(s,a)+\gamma\max Q(s',a') \]

        5. Move to the next room. 8. Display the final Q-table. 9. Stop.

Program:

```
# numpy stores the reward table and the learned table as grids of numbers.
import numpy as np

# Six rooms, numbered 0, 1, 2, 3, 4 and 5.
# A row is the room the agent is standing in.
# A column is the room the agent tries to enter.
# -1 means there is no door.
# 0 means the door is open but gives no reward.
# 100 means that door reaches the goal.
R = np.array([
```

```
    [-1, -1, -1, -1,  0, -1],
    [-1, -1, -1,  0, -1, 100],
    [-1, -1, -1,  0, -1, -1],
    [-1,  0,  0, -1,  0, -1],
    [ 0, -1, -1,  0, -1, 100],
    [-1,  0, -1, -1,  0, 100]
])

# Q starts as a grid of zeros with the same shape as R.
# The agent has not learned any move yet.
Q = np.zeros_like(R)

# gamma says how much a future reward is worth today. #
0.8 means a reward one step later counts as 80 percent.
gamma = 0.8

# The agent practices the whole trip this many times.
episodes = 1000

# One loop is one practice trip from a random room to the goal. for
_ in range(episodes):

    # Pick a starting room. randint(0, 6) gives 0, 1, 2, 3, 4 or 5.
state = np.random.randint(0, 6)

    # Keep walking until the agent is in room 5.
while state != 5:

        # Keep only the doors whose reward is 0 or 100.
# Those are the legal moves from the current room.
        actions = np.where(R[state] >= 0)[0]

        # Choose one legal door at random.
action = np.random.choice(actions)

        # In this example the next room number is the door number.
        next_state = action

        # Bellman update:
        # learned value = reward for this door
        # plus gamma times the best value already stored for the next room.
        Q[state, action] = R[state, action] + gamma * np.max(Q[next_state])

        # Step into the chosen room and repeat.
        state = next_state

# Print the finished Q-table.
# In a row, the biggest number is the best door from that room. print("Q-Table:\n")
print(Q)
```

Output:

```
C:\Users\burra\PycharmProjects\PythonProject1\.venv2\Scripts\python.exe C:\Users\burra\P
Q-Table:

[[  0   0   0   0  80   0]
 [  0   0   0  64   0 100]
 [  0   0   0  64   0   0]
 [  0  80  51   0  80   0]
 [ 64   0   0  64   0 100]
 [  0   0   0   0   0   0]]

Process finished with exit code 0
```

Result:

The Q-learning reinforcement learning model was implemented successfully. The agent learned action values through repeated interactions with the environment and rewards. The learned Q-table and the best action for each state were displayed.

Conclusion: The agent learns to move toward the goal by maximizing cumulative rewards. The Q-table represents the expected value of taking each action in each state.

10.Build a neural network using TensorFlow/Keras.

Aim:

To build and train a basic artificial neural network using TensorFlow/Keras to classify data and evaluate its performance.

Algorithm:

1. Start.
2. Import the required libraries: NumPy, TensorFlow/Keras, and Scikit-learn.
3. Load the dataset and separate the input features \(X\) and target labels \(y\).
4. Split the dataset into training and testing sets.
5. Normalize the input features if required.
6. Create a sequential neural network model.
7. Add an input layer, one or more hidden layers with ReLU activation, and an output layer with an appropriate activation function.
8. Compile the model using an optimizer, loss function, and evaluation metric.
9. Train the model using the training data for a specified number of epochs.
10. Evaluate the trained model using the testing data.
11. Display the test loss and accuracy.
12. Stop.

Pseudocode: Build a Neural Network Using TensorFlow/Keras

1. Start
2. Import Pandas, TensorFlow, Path, train_test_split, StandardScaler, and accuracy_score.
3. Set the TensorFlow random seed to 42.
4. Load the dataset nn_students.csv.
5. Select Hours_Studied, Attendance, and Assignments as input features \(X\).
6. Select Result as the target variable \(y\).
7. Convert the target labels: Fail = 0 and Pass = 1.
8. Split the dataset into 80% training data and 20% testing data.
9. Create a StandardScaler object.
10. Scale the training data using fit_transform().
11. Scale the testing data using transform().
12. Create a sequential neural network with: ○ Input layer with 3 features. ○ Hidden layer with 8 neurons and ReLU activation. ○ Hidden layer with 4 neurons and ReLU activation. ○ Output layer with 1 neuron and sigmoid activation.
13. Compile the model using the Adam optimizer, binary cross-entropy loss, and accuracy metric.
14. Train the model for 50 epochs using the training data and 20% validation split.
15. Predict the results for the testing data.
16. Convert predicted probabilities into class labels using a threshold of 0.5.
17. Calculate the model's accuracy by comparing predicted labels with actual labels.
18. Display the accuracy.
19. Stop.

Program:

```
# pandas reads the CSV file into a table.
import pandas as pd

# Path finds nn_students.csv in the same folder as this script.
from pathlib import Path

# tensorflow builds and trains the neural network. import
tensorflow as tf

# train_test_split keeps some students aside for a final test.
from sklearn.model_selection import train_test_split

# StandardScaler puts every column on a similar scale. from
sklearn.preprocessing import StandardScaler

# accuracy_score counts how many test students were predicted correctly.
from sklearn.metrics import accuracy_score

# Same starting weights on every run, so the accuracy does not change.
tf.random.set_seed(42)

# Read the 120-student file made for this experiment.
data = pd.read_csv(Path(__file__).with_name("nn_students.csv"))

# Three input numbers for each student.
X = data[["Hours_Studied", "Attendance", "Assignments"]]

# The answer column: Pass or Fail.
y = data["Result"]

# The network needs numbers. Fail becomes 0 and Pass becomes 1. y
= y.map({"Fail": 0, "Pass": 1})

# 80 percent of the rows are for training. 20 percent are for testing.
# random_state=42 keeps the same students in the test set every run.
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Learn the average and spread of each training column. scaler
= StandardScaler()

# Rescale the training rows. fit_transform learns the scale and applies it.
X_train = scaler.fit_transform(X_train)

# Rescale the test rows with the same scale. Do not fit again.
X_test = scaler.transform(X_test)

# Layers run in this order.
# Input: each student has 3 numbers.
# First hidden layer: 8 neurons, ReLU.
# Second hidden layer: 4 neurons, ReLU. #
Output: 1 neuron, sigmoid, a probability of Pass.
model = tf.keras.Sequential([
tf.keras.layers.Input(shape=(3,)),     tf.keras.layers.Dense(8,
```

```
activation="relu"),     tf.keras.layers.Dense(4,
activation="relu"),
    tf.keras.layers.Dense(1, activation="sigmoid")
])

# adam adjusts the weights.
# binary_crossentropy measures error for a 0-or-1 answer.
# accuracy is printed after every epoch.
model.compile(
optimizer="adam",
loss="binary_crossentropy",
    metrics=["accuracy"]
)

# Train for 50 passes over the training rows.
# validation_split=0.2 holds out the last 20 percent of those rows #
so each epoch can show a validation accuracy.
# verbose=1 prints one line per epoch.
history = model.fit(
X_train,     y_train,
epochs=50,
validation_split=0.2,
    verbose=1
)

# Each test student gets a probability of Pass, between 0 and 1.
y_pred = model.predict(X_test)

# A probability above 0.5 is called Pass (1). Otherwise it is Fail (0). y_pred
= (y_pred > 0.5).astype(int)

# Compare those labels with the real Result values.
accuracy = accuracy_score(y_test, y_pred)

# On nn_students.csv this prints 1.0, so every test student is correct.
print("Accuracy:", accuracy)
```

Output:

```
Epoch 1/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 1s 87ms/step - accuracy: 0.5658 - loss: 0.6804 - val_accuracy: 0.4000 - val_loss: 0.6851
Epoch 2/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 68ms/step - accuracy: 0.5658 - loss: 0.6639 - val_accuracy: 0.4000 - val_loss: 0.6747
Epoch 3/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step - accuracy: 0.5658 - loss: 0.6481 - val_accuracy: 0.4000 - val_loss: 0.6644
Epoch 4/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.5658 - loss: 0.6331 - val_accuracy: 0.4000 - val_loss: 0.6554
Epoch 5/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step - accuracy: 0.6184 - loss: 0.6197 - val_accuracy: 0.4500 - val_loss: 0.6472
Epoch 6/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 29ms/step - accuracy: 0.7105 - loss: 0.6072 - val_accuracy: 0.5500 - val_loss: 0.6397
Epoch 7/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 0.7895 - loss: 0.5961 - val_accuracy: 0.6000 - val_loss: 0.6326
Epoch 8/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - accuracy: 0.8289 - loss: 0.5867 - val_accuracy: 0.6500 - val_loss: 0.6256
Epoch 9/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 0.8816 - loss: 0.5779 - val_accuracy: 0.7500 - val_loss: 0.6184
Epoch 10/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step - accuracy: 0.8947 - loss: 0.5692 - val_accuracy: 0.8000 - val_loss: 0.6111
Epoch 11/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 70ms/step - accuracy: 0.9342 - loss: 0.5605 - val_accuracy: 0.9000 - val_loss: 0.6038
Epoch 12/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 0.9737 - loss: 0.5519 - val_accuracy: 1.0000 - val_loss: 0.5964
```

```
Epoch 14/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step - accuracy: 0.9737 - loss: 0.5348 - val_accuracy: 1.0000 - val_loss: 0.5815
Epoch 15/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9737 - loss: 0.5264 - val_accuracy: 1.0000 - val_loss: 0.5741
Epoch 16/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step - accuracy: 0.9868 - loss: 0.5180 - val_accuracy: 1.0000 - val_loss: 0.5667
Epoch 17/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9868 - loss: 0.5097 - val_accuracy: 1.0000 - val_loss: 0.5593
Epoch 18/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 1.0000 - loss: 0.5015 - val_accuracy: 1.0000 - val_loss: 0.5518
Epoch 19/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 1.0000 - loss: 0.4933 - val_accuracy: 1.0000 - val_loss: 0.5444
Epoch 20/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 71ms/step - accuracy: 1.0000 - loss: 0.4851 - val_accuracy: 1.0000 - val_loss: 0.5370
Epoch 21/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 44ms/step - accuracy: 1.0000 - loss: 0.4770 - val_accuracy: 1.0000 - val_loss: 0.5296
Epoch 22/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step - accuracy: 1.0000 - loss: 0.4689 - val_accuracy: 1.0000 - val_loss: 0.5222
Epoch 23/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step - accuracy: 1.0000 - loss: 0.4610 - val_accuracy: 1.0000 - val_loss: 0.5148
Epoch 24/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 1.0000 - loss: 0.4530 - val_accuracy: 1.0000 - val_loss: 0.5073
Epoch 25/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 45ms/step - accuracy: 1.0000 - loss: 0.4451 - val_accuracy: 1.0000 - val_loss: 0.4999
```

```
Epoch 26/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 1.0000 - loss: 0.4373 - val_accuracy: 1.0000 - val_loss: 0.4925
Epoch 27/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - accuracy: 1.0000 - loss: 0.4295 - val_accuracy: 1.0000 - val_loss: 0.4851
Epoch 28/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 26ms/step - accuracy: 1.0000 - loss: 0.4218 - val_accuracy: 1.0000 - val_loss: 0.4777
Epoch 29/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 1.0000 - loss: 0.4141 - val_accuracy: 1.0000 - val_loss: 0.4703
Epoch 30/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 1.0000 - loss: 0.4065 - val_accuracy: 1.0000 - val_loss: 0.4630
Epoch 31/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 1.0000 - loss: 0.3990 - val_accuracy: 1.0000 - val_loss: 0.4556
Epoch 32/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 1.0000 - loss: 0.3915 - val_accuracy: 1.0000 - val_loss: 0.4482
Epoch 33/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 1.0000 - loss: 0.3841 - val_accuracy: 1.0000 - val_loss: 0.4409
Epoch 34/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - accuracy: 1.0000 - loss: 0.3767 - val_accuracy: 1.0000 - val_loss: 0.4336
Epoch 35/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 50ms/step - accuracy: 1.0000 - loss: 0.3695 - val_accuracy: 1.0000 - val_loss: 0.4263
Epoch 36/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - accuracy: 1.0000 - loss: 0.3623 - val_accuracy: 1.0000 - val_loss: 0.4190
Epoch 37/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step - accuracy: 1.0000 - loss: 0.3551 - val_accuracy: 1.0000 - val_loss: 0.4118
```

```
Epoch 38/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 32ms/step - accuracy: 1.0000 - loss: 0.3481 - val_accuracy: 1.0000 - val_loss: 0.4046
Epoch 39/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step - accuracy: 1.0000 - loss: 0.3411 - val_accuracy: 1.0000 - val_loss: 0.3974
Epoch 40/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 40ms/step - accuracy: 1.0000 - loss: 0.3342 - val_accuracy: 1.0000 - val_loss: 0.3903
Epoch 41/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 1.0000 - loss: 0.3274 - val_accuracy: 1.0000 - val_loss: 0.3832
Epoch 42/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step - accuracy: 1.0000 - loss: 0.3207 - val_accuracy: 1.0000 - val_loss: 0.3762
Epoch 43/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 1.0000 - loss: 0.3141 - val_accuracy: 1.0000 - val_loss: 0.3692
Epoch 44/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 1.0000 - loss: 0.3076 - val_accuracy: 1.0000 - val_loss: 0.3623
Epoch 45/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step - accuracy: 1.0000 - loss: 0.3011 - val_accuracy: 1.0000 - val_loss: 0.3554
Epoch 46/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 1.0000 - loss: 0.2948 - val_accuracy: 1.0000 - val_loss: 0.3486
Epoch 47/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 1.0000 - loss: 0.2885 - val_accuracy: 1.0000 - val_loss: 0.3419
Epoch 48/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step - accuracy: 1.0000 - loss: 0.2824 - val_accuracy: 1.0000 - val_loss: 0.3352
Epoch 49/50
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 33ms/step - accuracy: 1.0000 - loss: 0.2764 - val_accuracy: 1.0000 - val_loss: 0.3287
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 31ms/step - accuracy: 1.0000 - loss: 0.2704 - val_accuracy: 1.0000 - val_loss: 0.3221
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 69ms/step
Accuracy: 1.0

Process finished with exit code 0
```

Result:

A basic neural network was built and trained using TensorFlow/Keras. The model was evaluated on the testing dataset, and its loss and accuracy were obtained to measure its classification performance.